Digital Design and Computer Architecture

A complete overview of the materials taught in the Digital Design and Computer Architecture course of the Basisjahr's second semester at ETH Zürich.

Erik Girdauskas

Table of Contents

Boolean Algebra and Basic Logic 1
Combinational Logic Blocks 4
Sequential Logic 7
Memory Types 9
Finite State Machines 10
Timing, Delay, and Clocking 13
Combinational Timing 14
Sequential Timing 15
Verilog 17
Sequential Logic 19
FSMs, Testbenches, and Simulation 20
Von Neumann Model 21
LC-3 - Expanded Example 25
Data Movement and Addressing Modes 25
Control Instructions and Branching 26
The MIPS ISA 27
MIPS Instruction Formats 28
Basic Instructions 29
Control Instructions and Advanced Concepts 30
Procedures 31
Stack 32
Recursion 32
Microarchitecture 33
Single-Cycle CPU 34
Multicycle CPU 37
Pipelining 39
Data Hazards 41
Branch Hazards 43
Exceptions and Precise State 44
Out-of-Order Execution 45
Reorder Buffer and Register Renaming 46
Dataflow 47
Tomasulo's Algorithm 47
Superscalar Execution 51
Branch Prediction 52
Dynamic Branch Prediction 54
VLIW 56
Systolic Arrays 57

SIMD 58
Array Processors 58
Vector Processors 59
GPUs 62
Memory Hierarchy 66
Caches 69
Virtual Memory 73
Optional x86 Section 82
Additional Exam Problems 84

Color code: major topics, important stuff

by Erik Girdauskas

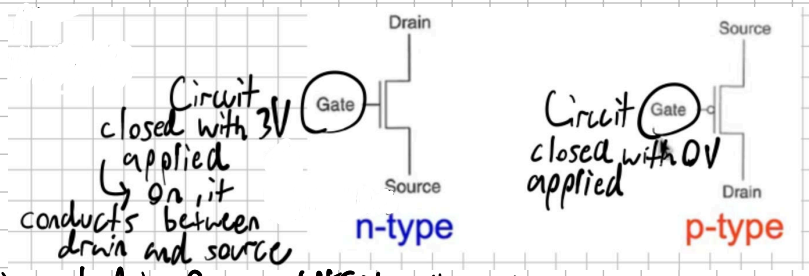
semester recap - DDCA

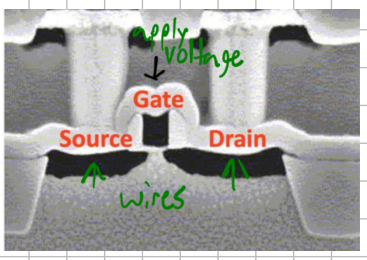
foreword: if there is one subject to trust me on, it's this! the sheet is based on my reading notes + the lectures

Basics/Boolean Algebra:

Transistors: The most basic building blocks of computers, they are mostly (only?) implemented with MOS → metal, oxide (as insulator), semiconductors

→ They are used to build logic gates, don't worry about constructing anything with them for the exam though!





→ 3V is a stand-in for any HIGH voltage here

→ think of n-type transistors as active high and active low components

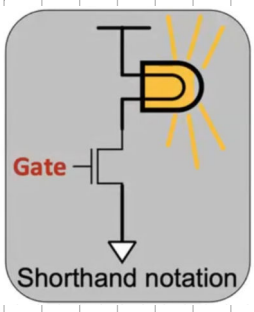
→ exercise: the 7-segment display in our FPGA displays a value when its transistors are driven to a LOW voltage. What transistors does it use?

Logic Gates:

→ They are basic modules made from transistors and boolean algebra is performed on them

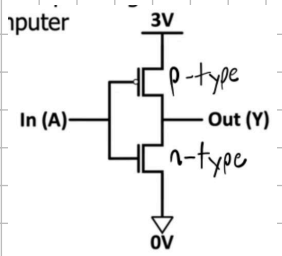
→ the term CMOS stands for complementary MOS, describing the use of both p-type and n-type transistors in larger structures like gates

general structure of gate schematic →

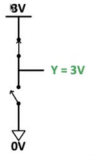


NOT gate:

(logical inverter)



Truth Table (these all use truth tables):

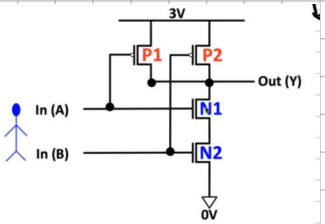


A	P	N	Y
0	ON	OFF	1
1	OFF	ON	0

↖ example path of 0V being passed as input

NAND gate:

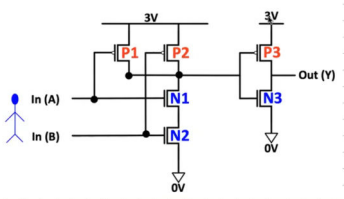
(not and)



TT:

A	B	P1	P2	N1	N2	Y
0	0	ON	ON	OFF	OFF	1
0	1	ON	OFF	OFF	ON	1
1	0	OFF	ON	ON	OFF	1
1	1	OFF	OFF	ON	ON	0

AND gate:



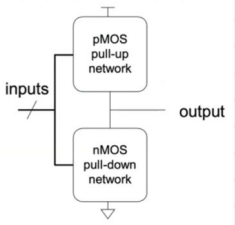
TT:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

→ comment: see how we built this from a NAND gate and an inverter? This saves us transistors compared to building it from scratch

→NAND and NOR are logically complete because you can build any Boolean expression using those two gates

CMOS gate structure:



→ Example: in XOR, only one network should be on, if both are on, there's a short-circuit, if neither are, an undefined floating output is sent out

→ pMOS only carries 1's (HIGH voltage) well, nMOS only carries 0's (LOW voltage) well

→ this means that with pMOS, passing a logical LOW causes voltage loss (other way around for nMOS)

→ it's also the reason we need both (CMOS)

Electrical stuff (maybe exam relevant?)

→ you can compare running something in series electrically to a logical AND and electrical parallel to a logical OR

→ **Latency:** Series puts more resistance on the wire carrying it, and as resistance determines latency, series is slower

→ This is why practically, as little series as possible is used

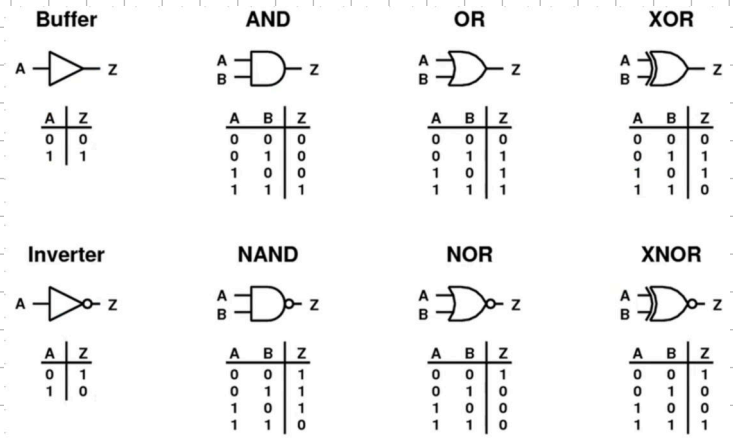
→ **Dynamic Power Consumption:** power to charge a transistor while the signal changes (inversion): $C \cdot V^2 \cdot f$ (Capacitance ↓, Voltage ↓, Charge frequency ↓ of capacitor)

→ **Static Power Consumption:** power used while signals do not change: $V \cdot I_{leakage}$ ← current that keeps flowing, hard to minimize

→ **Energy Consumption:** Power • Time

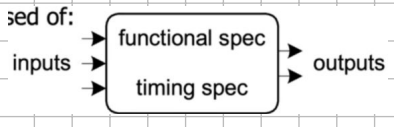
→ at no point is the power used meant to exceed the Thermal Design Product (TDP, max. intended power)

Diagram for **Gates and their Truth Tables** (from now on, we won't use the transistor-level schematic)



Circuits:

Structure:



Functional spec.: putting outputs in terms of inputs (using the math. def. of function)

→ this uses Boolean algebra, which is mostly repetition from DiskMath

Axioms and laws: (+ replaces v from DM, • replaces ∧, $\overline{A}$ replaces ¬A)

All axioms are given in **dual form:**

- A dual of a Boolean expression is derived by replacing
  - Every AND operation with... an OR operation
  - Every OR operation with... an AND
  - Every constant 1 with... a constant 0
  - Every constant 0 with... a constant 1
  - But don't change any of the literals or play with the complements!

Example $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$

→ $a + (b \cdot c) = (a + b) \cdot (a + c)$

(B refers to the entire Boolean expression)

Formal version

1. *B* contains at least two elements, *0* and *1*, *such that* $0 \neq 1$
2. *Closure* $a,b \in B$,
   (i) $a + b \in B$
   (ii) $a \cdot b \in B$
3. *Commutative Laws*: $a,b \in B$,
   (i) $a + b = b + a$
   (ii) $a \cdot b = b \cdot a$
4. *Identities*: $0, 1 \in B$
   (i) $a + 0 = a$
   (ii) $a \cdot 1 = a$
5. *Distributive Laws*:
   (i) $a + (b \cdot c) = (a + b) \cdot (a + c)$
   (ii) $a \cdot (b + c) = a \cdot b + a \cdot c$
6. *Complement*:
   (i) $a + \bar{a} = 1$
   (ii) $a \cdot \bar{a} = 0$

		Dual	
Operations with 0 and 1:			
1. $X + 0 = X$		1D. $X \cdot 1 = X$	AND, OR with identities gives you back the original variable or the identity
2. $X + 1 = 1$		2D. $X \cdot 0 = 0$	
Idempotent Law:			
3. $X + X = X$		3D. $X \cdot X = X$	AND, OR with self = self
Involution Law:			
4. $(\bar{X}) = X$			double complement = no complement
Laws of Complementarity:			
5. $X + \bar{X} = 1$		5D. $X \cdot \bar{X} = 0$	AND, OR with complement gives you an identity
Commutative Law:			
6. $X + Y = Y + X$		6D. $X \cdot Y = Y \cdot X$	Just an axiom…
Associative Laws:			
7. $(X + Y) + Z = X + (Y + Z)$ $= X + Y + Z$		7D. $(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$ $= X \cdot Y \cdot Z$	Parenthesis order does not matter
Distributive Laws:			
8. $X \cdot (Y + Z) = (X \cdot Y) + (X \cdot Z)$		8D. $X + (Y \cdot Z) = (X + Y) \cdot (X + Z)$	Axiom
Simplification Theorems:			
9. $X \cdot Y + X \cdot \bar{Y} = X$		9D. $(X + Y) \cdot (X + \bar{Y}) = X$	Useful for simplifying expressions
10. $X + X \cdot Y = X$		10D. $X \cdot (X + Y) = X$	
11. $(X + \bar{Y}) \cdot Y = X \cdot Y$		11D. $(X \cdot \bar{Y}) + Y = X + Y$	

DeMorgan's Law:

12. $\overline{(X + Y + Z + \cdots)} = \bar{X}.\bar{Y}.\bar{Z}. \ldots$

12D. $\overline{(X.Y.Z. \ldots)} = \bar{X} + \bar{Y} + \bar{Z} + \ldots$

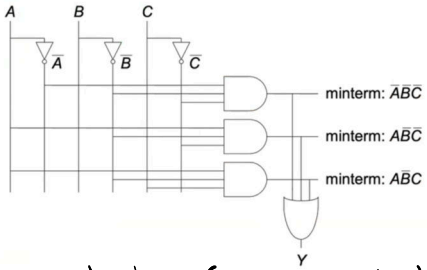
Canonical function forms:

Sum of Products (SoP, identical to DNF from DiskMath):

→ two-level logic: at most two levels of gates → no path goes through more than 2 gates (SoP or Product of Sums (PoS))

→ all Boolean equations can be put into SoP and PoS

- **SOP (sum-of-products) leads to two-level logic**
- Example: $Y = (\bar{A} \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot C)$



→ deriving it from a truth table: find every entry that evaluates to 1, make these an implicant (connected with •), then make a disjunction of implicants (connect with +)

→ each implicant here is a minterm (product of all input variables)

→ standard notation: enumerate over minterms based on the binary number created by the input pattern

→ Example: $\Sigma m(0,6,7)$

PoS: (identical to CNF)

→ deriving it from a truth table: take each expression that evaluates to false, disjoin them (connect with +, this makes them maxterms (sum of all input variables) and conjoin the maxterms (using •)

→ standard notation: similar to SoP, but take the product of maxterms (like: $\prod M(0,6,7)$

Conversion rules:

1. **Minterm to Maxterm conversion:**
   rewrite minterm shorthand using maxterm shorthand
   replace minterm indices with the indices not already used
   E.g., $F(A,B,C) = \sum m(3,4,5,6,7) = \prod M(0,1,2)$
2. **Maxterm to Minterm conversion:**
   rewrite maxterm shorthand using minterm shorthand
   replace maxterm indices with the indices not already used
   E.g., $F(A,B,C) = \prod M(0,1,2) = \sum m(3,4,5,6,7)$
3. **Expansion of F to expansion of $\bar{F}$:**

E.g., $F(A,B,C) = \sum m(3,4,5,6,7) \longrightarrow \bar{F}(A,B,C) = \sum m(0,1,2)$

$= \prod M(0,1,2) \longrightarrow = \prod M(3,4,5,6,7)$

4. **Minterm expansion of F to Maxterm expansion of $\bar{F}$:**
   rewrite in Maxterm form, using the same indices as F

E.g., $F(A,B,C) = \sum m(3,4,5,6,7) \longrightarrow \bar{F}(A,B,C) = \prod M(3,4,5,6,7)$

$= \prod M(0,1,2)$

Combinational Blocks:

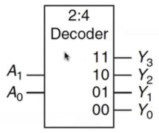
→ constructed using gates, they are a layer of abstraction above gates, so gate-level details are hidden

→ Note: Combinational Logic has no memory

Decoder: Has a separate output for every possible input pattern, n inputs, 2^n outputs

2-to-4 decoder:

A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



→ oh you will see how it comes up in a CPU

In Verilog:

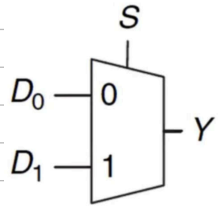
```
module decoder #(parameter   N = 3)
                (input       [N-1:0]    a,
                 output reg  [2**N-1:0] y);

  always @ (*)
    begin
      y = 0;
      y[a] = 1;
    end
endmodule
```

set output indexed at numerical value of input a, to 1

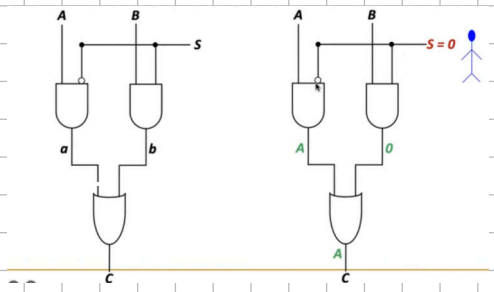
Multiplexer (MUX): Core idea: select one of N inputs to mirror to the output, decision is made by the log2N-bit control input:

2-to-1 MUX:

S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



(gate-level):



In Verilog (2-to-1 MUX):

```
Verilog
module mux2
  #(parameter width = 8)
   (input  [width-1:0] d0, d1,
    input              s,
    output [width-1:0] y);
  assign y = s ? d1 : d0;
endmodule
```

default value

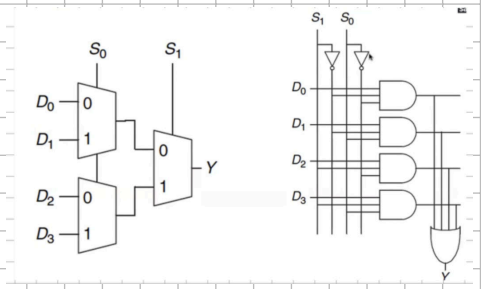
depends on width

we are passing in variable to decide module's width

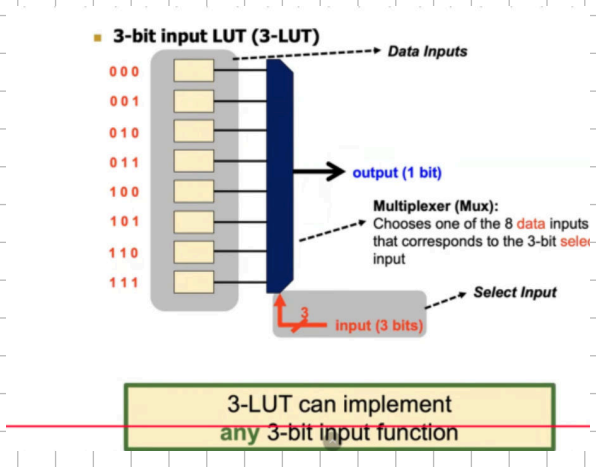
Simpler TT:

S	C
0	A
1	B

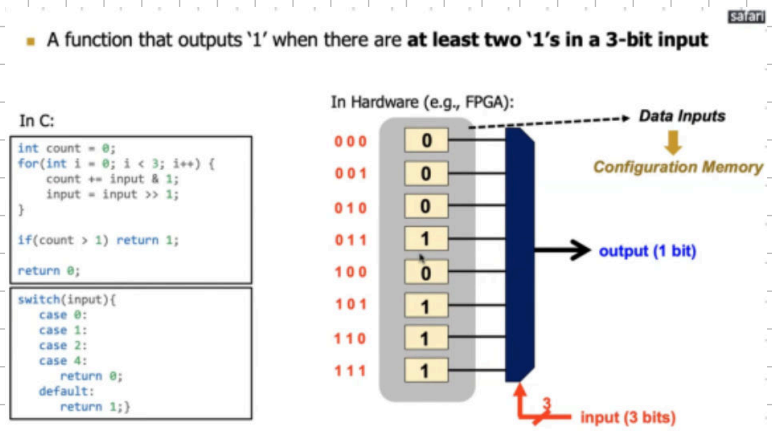
4-to-1 mux:



→ Lookup Tables (LUT), which our FPGAs use prominently for reprogrammable comb. logic, are implemented as trees of MUXes



3-LUT can implement any 3-bit input function



4

Full Adder: our way of performing bitwise addition on two inputs (A, B) with a carry C

gate-level:

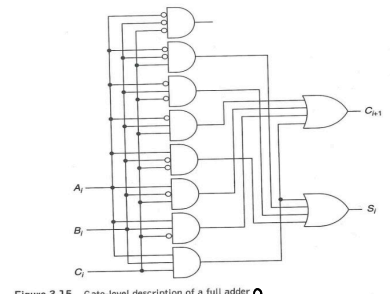


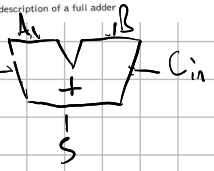
Figure 3.15 Gate-level description of a full adder

→ AND gates for each combination of "current-bit" inputs, OR gates for "current-bit" output S_i and "next-bit" carry C_{i+1}

TT:

a_i	b_i	$carry_i$	$carry_{i+1}$	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Block Diagram:



for **multi-bit addition**, the most efficient CL (comb. logic) block is the **Carry Lookahead Adder:** (optional explanation)

↳ in H&H Chapter 5, you see the Ripple-Carry Adder, which has more latency as the addition is done sequentially from the MSB (most significant bit) to the LSB (least significant bit), so the delay Tripple grows with the number of bits (linearly)

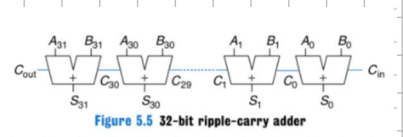


Figure 5.5 32-bit ripple-carry adder

The Carry Lookahead Adder (CLA) eliminates this delay:

→ the adder is **divided** into blocks

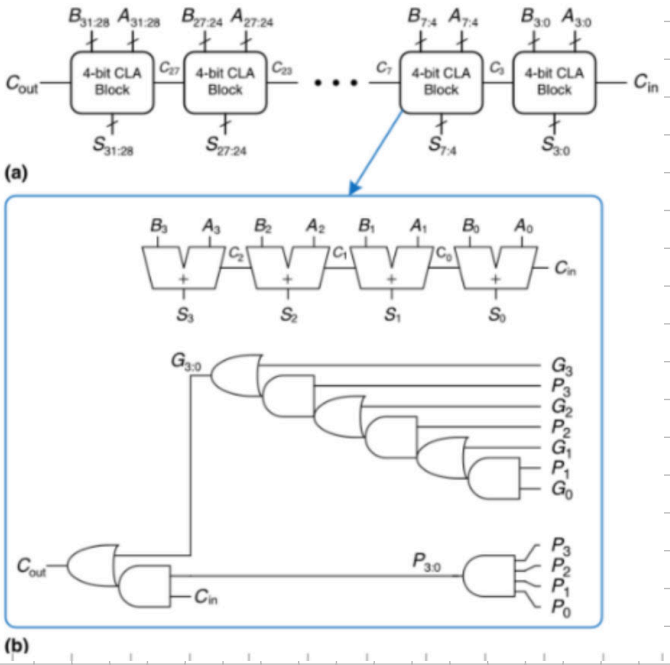
→ circuitry can quickly determine C_{out} of a block with known C_{in}

→ **Generate (G) and Propagate (P)** signals describe how C_{out} is determined

→ the i-th column of the CLA **generates** a carry-out if it produces one independent of C_{in} → C_i is generated if $A_i = 1$ and $B_i = 1 \Rightarrow A_i B_i$

→ it **propagates** a carry-out if $A_i = 1$ or $B_i = 1 \Rightarrow A_i + B_i$

→ the i-th column **generates** c_i if it generates a carry or propagates a carry in $(P_i C_{i-1}) \Rightarrow C_i = A_i B_i + (A_i + B_i) C_{i-1} = G_i + P_i C_{i-1}$



→ our definitions from now extend to **multi-bit blocks:** a block generates a carry if it produces one independent of C_{in} and propagates one if a carry is produced under the condition of $C_{in} = 1$

→ G and P extend to $G_{i:j}$ and $P_{i:j}$ for the block $i:j$

→ **pattern:** Block generates a carry if most sig. column generates one or the most sig. column generates one and the next sig. column generates one ...

→ $G_{3:0} = G_3 + P_3(G_2 + P_2(G_1 + P_1 G_0))$

→ **it propagates one** if all columns propagate: $P_{3:0} = P_3 P_2 P_1 P_0$

→ Carry formula: $C_{i:j} = G_{i:j} + P_{i:j} C_j$

→ single-bit and **block generate/propagate** is computed simultaneously

→ **Critical path** (defined later): Compute $G_{3:0}$ and G_0 in the first block, then advance sequentially

→ **timing definition** (also formally introduced later): $t_{CLA} = t_{pg} + t_{pg\_block} + \left(\frac{N}{k} - 1\right) \cdot t_{AND\_OR} + k \cdot t_{FA}$ (for N bits in k-bit blocks)

(t_{CLA}: total delay; t_{pg}: individual P,G gates (single AND/OR construct); t_{pg_block}: signals for k-bit block; t_{AND_OR}: delay C_{in} to C_{out} through AND/OR logic of k-bit block; t_{FA}: k full adders)

Verilog adder:

```
Verilog
module adder #(parameter N = 8)
              (input  [N-1:0] a, b,
               input          cin,
               output [N-1:0] s,
               output         cout);

  assign {cout, s} = a + b + cin;
endmodule
```

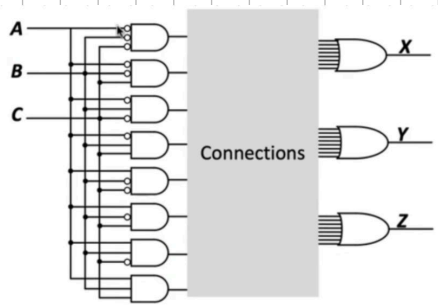
bundled parameters → {a, b}

I will just continuously leave these on the notes to comb. blocks

→ we can determine dependencies (carries) based off just our block's information, allowing for parallelization
→ keep in mind that there are always tradeoffs: CLA is faster than ripple, but requires more complex hardware and area

Programmable Logic Array (PLA):

→ Universal structure allowing execution of any n-input logic function



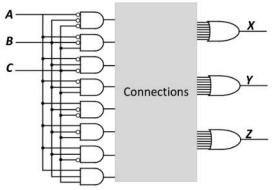
Amount of AND gates → amount of possible SOP minterms → 2^n

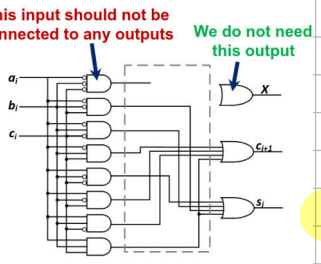
OR gates → amount of output columns (we call it M) ($\leq 2^n$)

programming them: program connections from AND gates to OR gates

→ you can base this on truth tables

Example:





Truth table of a full adder

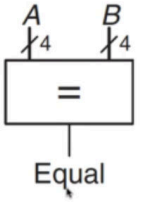
a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

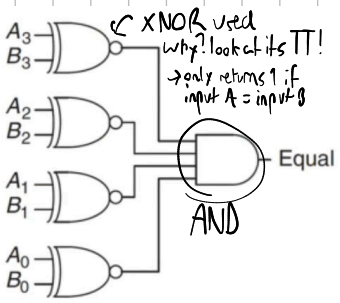
Logical Completeness: if a set of gates is logically complete, you can implement any logical function with it

→ {AND, OR, NOT} is log. complete → Example: PLA

→ {NAND, NOR} are also log. complete

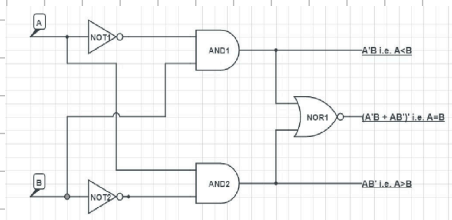
Comparator:



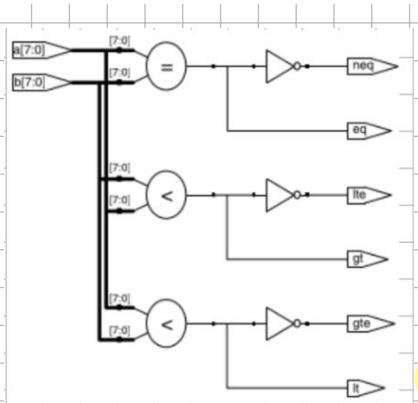


(our first lab!)

Magnitude comparator:



N-bit magnitude comparator in Verilog:



synthesizes to

this style is known as behavioral Verilog, we will go into it in a bit

```
module comparators # (parameter N = 8)
                    (input [N-1:0] a, b,
                     output        eq, neq,
                     output        lt, lte,
                     output        gt, gte);

  assign eq  = (a == b);
  assign neq = (a != b);
  assign lt  = (a <  b);
  assign lte = (a <= b);
  assign gt  = (a >  b);
  assign gte = (a >= b);
endmodule
```

ALU (Arithmetic Logic Unit)

→ this is crucial to CPUs

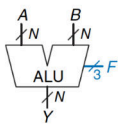


Figure 5.14 ALU symbol

Table 5.1 ALU operations

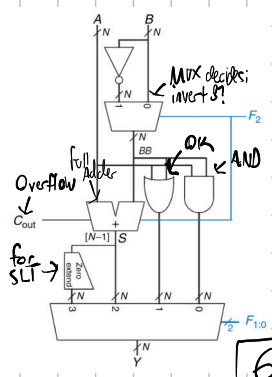
$F_{2:0}$	Function
000	A AND B
001	A OR B
010	A + B
011	not used
100	A AND $\overline{B}$
101	A OR $\overline{B}$
110	A – B
111	SLT

→ operations vary depending on your microarchitecture

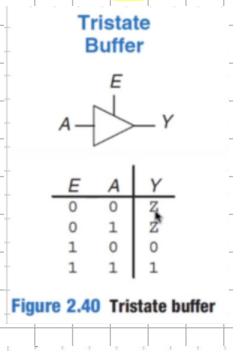
→ our lab one had XOR and NOR

aside: 2's complement → negative binary numbers start with 1, zero is always a N-bit zero
→ this is how ALU's do math usually: convert a positive like 0101 (5) to negative → invert bits, then add 1 → 1011 (-5)

Implementation:



Tri-State Buffer:

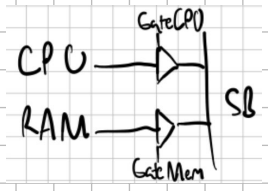


E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1

Figure 2.40 Tristate buffer

→ Z is a floating value that is neither 1 nor 0 (more later)

→ example usage: a shared bus → beautiful diagram:



→ when CPU in use, we set GetMem to 0, it pushes Z to the bus, which doesn't interfere with the CPU value

Simplify TT:

E	Y
0	Z
1	A

cool usage: MUX using tri-state buffers:

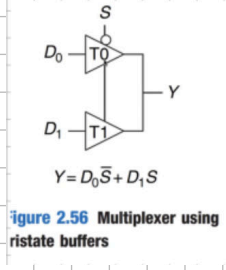
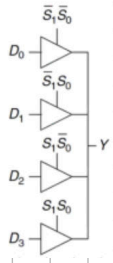


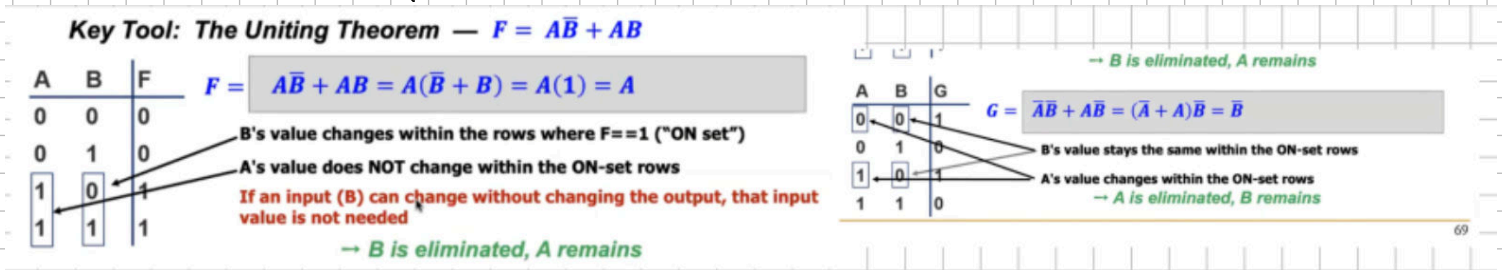
Figure 2.56 Multiplexer using tristate buffers



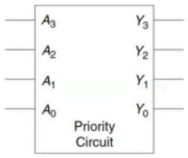
Logic Simplification:

→ why? we want to reduce the number of gates/inputs to save space/latency

→ Uniting Theorem: find "ON" set of function, if an input can change without changing the input, its value is not needed



Priority Circuit: Inputs: requestors, Output: who gets priority:



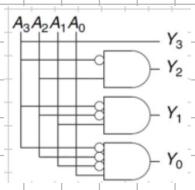
A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0

→ simplify:

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	X	0	0	1	0
0	1	X	X	0	1	0	0
1	X	X	X	1	0	0	0

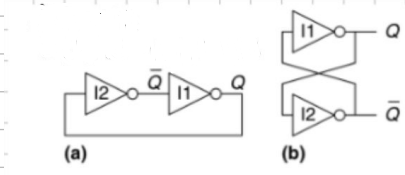
Figure 2.29 Priority circuit truth table with don't cares (X's)

circuit:



Sequential Logic:

Cross-Coupled Inverter:



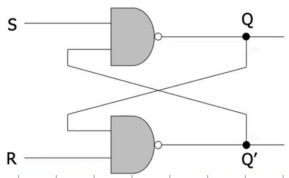
→ Q and Q̄ depend on each other

Case 1: Q = 0; I_2 receives false → Q̄ turns true, I_1 receives false → Q false...

Case 2 analogous → the circuit has 2 stable states (bistable)

→ Metastable state: both outputs halfway between 0 and 1 (explained later)

→ we have no control mechanism, so this concept is expanded into the R-S latch:



Input		Output
R	S	Q
1	1	Q_{prev}
1	0	1
0	1	0
0	0	Forbidden

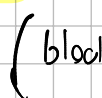
→ instead of inverters, we cross-coupled NAND gates (book uses NOR)

→ Data stored at Q, its inverse at Q'

→ R and S are control inputs (quiescent (idle) state: both are 1)

→ S (set): you set by driving S to 0 and not resetting (keep R at 1)

→ R (reset): you reset by driving R to 0 and not setting (keep S at 1)

(block diagram: 

)

→ S and R should not both be at 0. why?

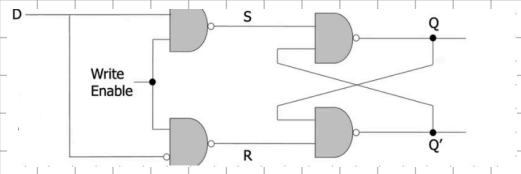
→ logically: you cannot both set and reset at once, this also causes both Q and Q' to be 1 (breaking the invariant there: meaning of Q)

→ practically: if both Q and Q' are set to 1 at the same time, Q and Q' start to oscillate (metastability, values depend on each other)

→ they would just start wildly switching between 1 and 0 because Q cannot equal Q'

→ the time this takes to resolve varies (seen later)

→ The gated D latch guarantees correct operation:



Input		Output
WE	D	Q
0	0	Q_{prev}
0	1	Q_{prev}
1	0	0
1	1	1

→ basic version uses a clock instead of WE

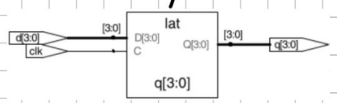
(block diagram: 

)

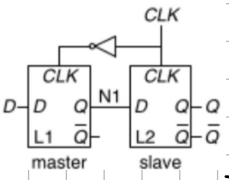
→ D is our only real input (data), if WE is enabled, Q takes the value of D

Verilog latch →

```
module latch (input        clk,
              input  [3:0] d,
              output reg [3:0] q);
  always @ (clk, d)
    if (clk) q <= d;
endmodule
```



D flip-flop: (I'll use the clock version, you can imagine CLK as WE for now)



→ The reason we use this problematic terminology is for the following analogy: "slave copies master"

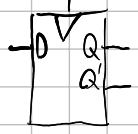
→ L_1 and L_2 are D latches, L_1 is the master, L_2 is the slave

→ CLK = 0 → L_1 is transparent (taking in the value from D), L_2 opaque (keeping its value)

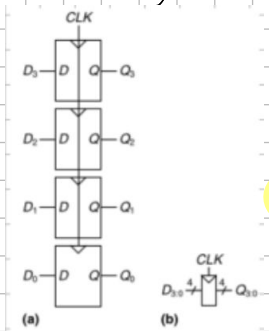
→ CLK = 1 → L_1 is opaque, L_2 is transparent (N_1 is cut off from D and takes the value stored in Q)

→ in action: L_1 copies to L_2 at the rising edge (CLK goes from 0 → 1) and remembers its state otherwise

Block diagram:



N-bit register: bank of N flip-flops



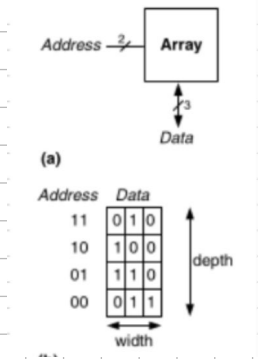
→ they share CLK and all flip-flops are updated at the rising edge

Adjacent concepts:

Enabled flip-flop → WE decides if Q is written on the rising edge

Resettable flip-flop → RESET input forces Q = 0
↳ active low input → something happens after it is at 0

Memory arrays: shared properties of all memories:



→ memory is organized as a 2D array of memory cells

→ a program reads/writes the contents of an array row

→ rows are addressable → an array with N-bit addresses and M-bit data has 2^N rows and M columns

→ each row is called a word → there are 2^N M-bit words

→ depth → #rows, width → #columns (word size), size → depth · width

→ memory arrays are built as arrays from bit cells

→ each bit cell stores 1 bit of data and is connected to both the wordline and bitline

→ for each combination of address bits, the memory asserts a single wordline that activates all bit cells in the row

→ wordline is HIGH → stored bit is transferred to or from the bitline

→ else, the bitline is not connected to the bit cell

→ usually, the bitline is at Z to read, when the wordline is ON, it is set to ON → this connects bitline to stored bit

→ this causes the bitline to overpower and rewrite the bit cell's data

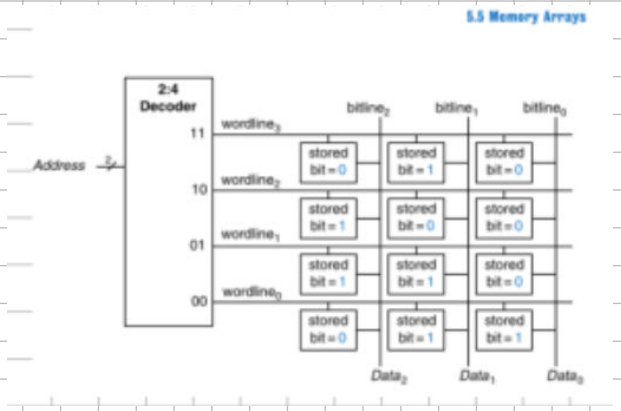
Memory Arrays:

→ read: wordline asserted, bit cells then drive their lines H/L

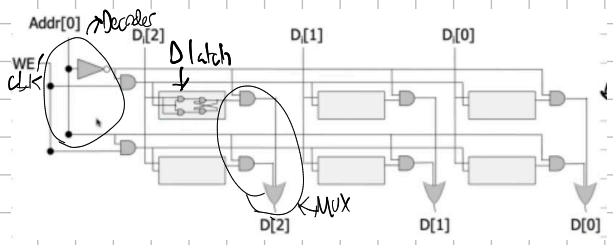
→ write: drive bitlines H/L, then assert wordline to store the new values

→ ports: all memories have ≥ 1 port for read and write access each

→ a multiported memory allows simultaneous reading/writing

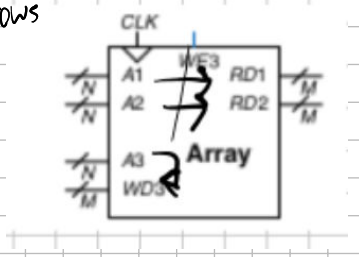


The lectures used a slightly different design:



→ here, the point is to set individual cells rather than rows

→ decoder to decide row, MUX to decide current row → select input



Memory types: (NOTE: this is part 1, part 2 will come around W11-W12 → caches, memory banking, ...)

DRAM: a bit is stored as the presence or absence of charge in a capacitor

→ nMOS transistor behaves as switch between capacitor and bitline

→ wordline asserted → nMOS turns ON → bit value put on/taken from bitline

→ it is not driven high/low by a transistor tied to VDD (voltage needed for HIGH) or GND (ground, voltage for LOW) (dynamic RAM)

→ on read: data value driven from capacitor to bitline, write: opposite

→ charge leaks over time → DRAM must be reset every few ms when not read

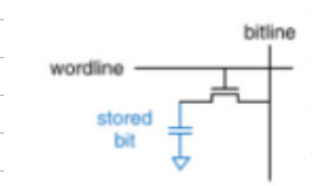


Figure 5.44 DRAM bit cell

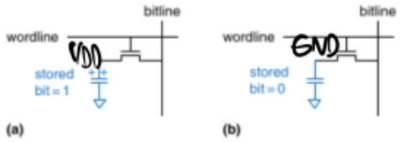


Figure 5.45 DRAM stored values

SRAM: static, does not need to be refreshed

→ data bit stored in cross-coupled inverter with outputs bitline, $\overline{\text{bitline}}$ (as Q, $\overline{Q}$)

→ wordline asserted → both nMOS turn on, values are either set or gotten

→ if noise degrades charge, cross-coupled inverters restore values

→ latency and throughput depend on array size

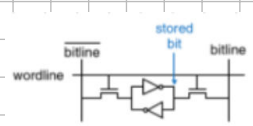


Figure 5.46 SRAM bit cell

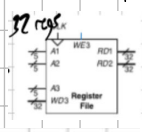
Memory Type	Transistors per Bit Cell	Latency
flip-flop	~20	fast
SRAM	6	medium
DRAM	1	slow

important, DRAM is cheapest and slowest

9

Register files: systems hold temporary values there, it's a small, multiport SRAM array, cheaper than flip-flops

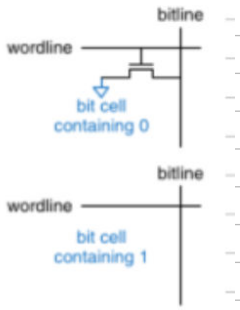
→ you will see that caches are basically fully SRAM

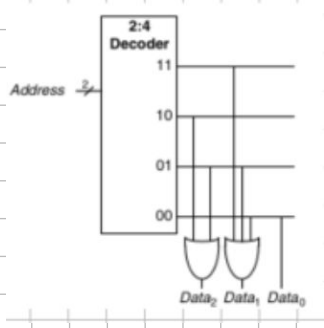


ROM: a bit is the presence or absence of a transistor

→ reading a cell: when the wordline of a row is ON, pull the bitline HIGH (weakly)

→ if a transistor exists, the corresponding bitline is driven LOW (absence means binary 0)





→ a ROM bit cell is a combinational circuit (no state to remember)

→ any ROM can be constructed from a decoder and NOT gates

→ ROM can perform any two-level logic function (→ recap: level:

→ traditional ROM is hardcoded in manufacturing

→ if the transistors are always there, but disconnectible to ground, we have Programmable ROM (PROM)

→ if instead of transistors, we use floating gates, we have (Electrically) Erasable PROM (EEPROM/EPROM depending on technical details)

In Verilog:

```
module ram# (parameter N = 6, M = 32)
            (input           clk,
             input           we,
             input  [N-1:0]  adr,
             input  [M-1:0]  din,
             output [M-1:0]  dout);

  reg [M-1:0] mem [2**N-1:0];

  always @ (posedge clk)
    if (we) mem[adr] <= din;

  assign dout = mem[adr];
endmodule
```

rows, word length

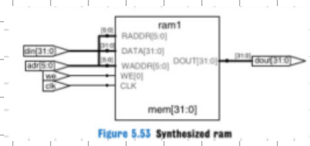


Figure 5.53 Synthesized ram

```
module rom (input      [1:0] adr,
            output reg [2:0] dout);

  always @ (adr)
    case (adr)
      2'b00: dout <= 3'b011;
      2'b01: dout <= 3'b110;
      2'b10: dout <= 3'b100;
      2'b11: dout <= 3'b010;
    endcase
endmodule
```

Back to sequential circuits:

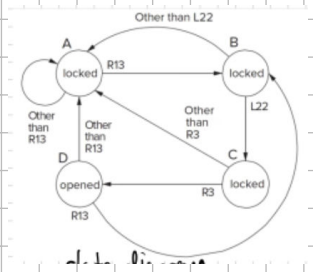
State: this remembers past events and contributes to the output depending on current inputs

Analogy: Combinational lock:

State A: lock not open, 0/3 inputs
B: locked, 1/3 complete
C: locked, 2/3 complete
D: unlocked

output: need to be completed in order, else return to A

→ state diagram for combination R13, L22, R3
(it's one of the locks you turn left/right)



Asynchronous vs. synchronous:

→ async.: immediate state transition after input, state transitions are synchronized by nothing

→ sync.: state transitions at fixed times

→ done in most modern computers, it is much simpler to design, but input is delayed (usually by picoseconds today?)

→ in our course, we always use a clock to synchronize (you've seen this in the memory chapter already)

Clock:

→ state only changes at the rising and falling edges in a clock-controlled circuit

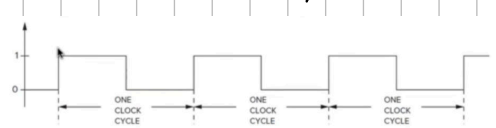


Figure 3.28 A clock signal.

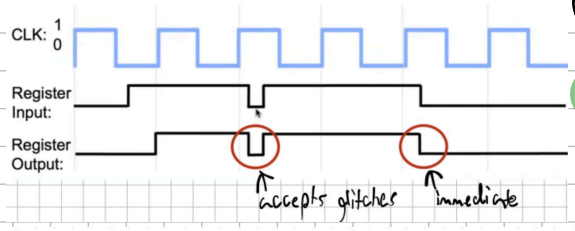
10

→ we use the clock to trigger transitions across many elements in sync. circuit

→ during a clock cycle, combinational logic blocks compute the next state (also known as next-state logic)

→ in the timing chapter, you will see how important it is for a clock cycle to be longer than any combinational (computing) delay

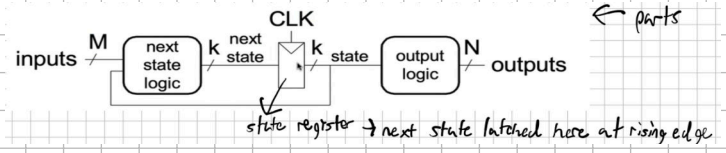
reason why we use flip-flops and not latches in sequential circuits:



(this is for a D latch)

FSM: discrete time-model of a stateful system

it contains:
- a finite numbers of states
- a finite number of external inputs
- a finite number of external outputs
- a specification of all state transitions
- a specification of how external outputs are determined



← parts

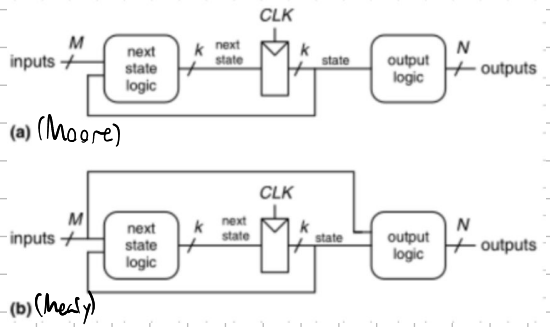
→ next state latched here at rising edge

→ The state register(s) store the current state and output the next state at the rising edge (only sequential circuit)

→ Combinational circuits: Next state logic and output logic

Moore machines: outputs depend on current state only

Mealy machines: outputs also depend on current inputs



FSM design (crucial for exam!):

1. Identify your inputs/outputs
2. Draw your state transition diagram
   - ↳ start with the reset state and what is caused by it
   - → add states and transitions
   - → this is similar to programming
     - → FSMs have a sequential control flow (conditionals, jumps)
     - → if/else construct controlled by inputs → outputs controlled by state or inputs
3. Moore machine: write state transition table, write output table

   Mealy machine: write state transition and output table (combined)
4. Select state encodings:

   Binary (full) encoding: Use minimum possible number of bits → $\log_2$(#states) bits → Example: 00, 01, 10, 11
   - → minimizes # flip-flops, not necessarily output logic

   One-hot encoding: one bit for every state → #states bits needed → example: 0001, 0010, 0100, 1000
   - → simple, automatable, doesn't need much next state logic, but inefficient for flip-flops

   Output encoding: directly encode outputs in state

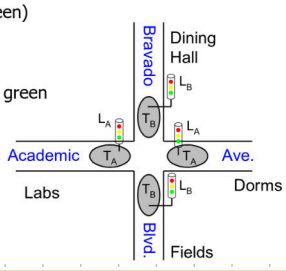
→example: 001 (green), 010 (yellow), 100 (red), 110 (red/yellow) for a traffic light (only works for Moore type)

5. Write Boolean equations for next state and output logic

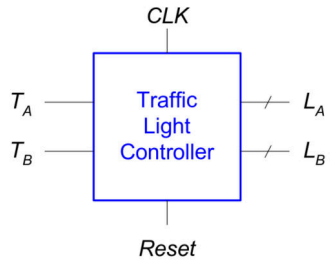
6. Sketch the circuit (this is formally you won't have to do this)

Simple example:

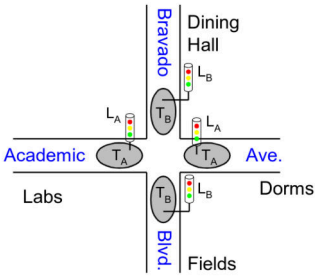
- "Smart" traffic light controller
  - **2 inputs**:
    - Traffic sensors: T_A, T_B (TRUE when there's traffic)
  - **2 outputs**:
    - Lights: L_A, L_B (Red, Yellow, Green)
  - State changes every 5 seconds
    - Except if green and traffic, stay green

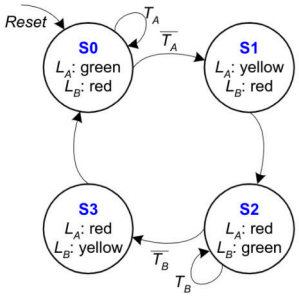


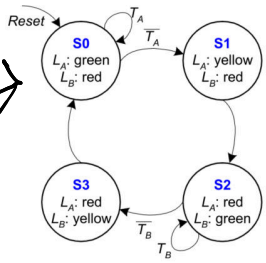
- **Inputs:** CLK, Reset, T_A, T_B
- **Outputs:** L_A, L_B



- **Moore FSM:** outputs labeled in each state
  - **States:** Circles
  - **Transitions:** Arcs

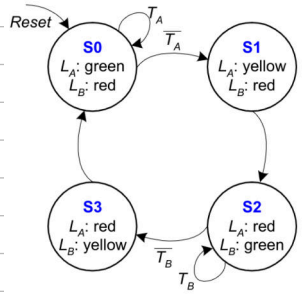






Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

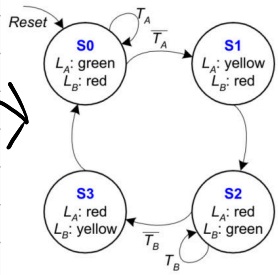
State	Encoding
S0	00
S1	01
S2	10
S3	11



Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

State	Encoding
S0	00
S1	01
S2	10
S3	11

$$S'_1 = (\overline{S_1} \cdot S_0) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B}) + (S_1 \cdot \overline{S_0} \cdot T_B)$$

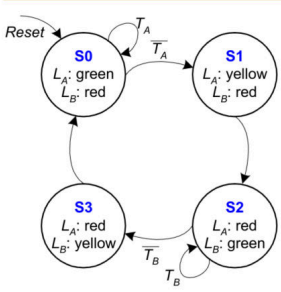


Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

State	Encoding
S0	00
S1	01
S2	10
S3	11

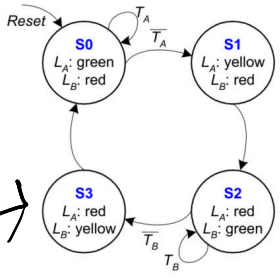
$S'_1 = S_1 \text{ xor } S_0$ **(Simplified)**

$$S'_0 = (\overline{S_1} \cdot \overline{S_0} \cdot \overline{T_A}) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B})$$



Current State		Outputs	
S_1	S_0	L_A	L_B
0	0	green	red
0	1	yellow	red
1	0	red	green
1	1	red	yellow

Output	Encoding
green	00
yellow	01
red	10



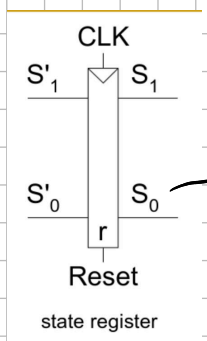
Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

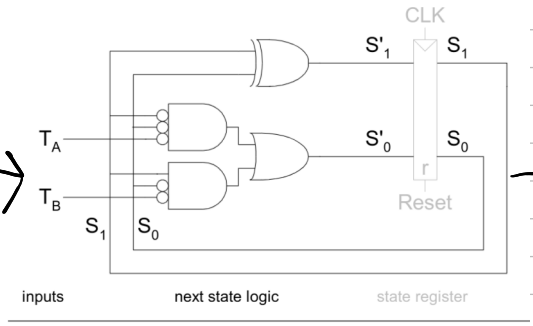
Output	Encoding
green	00
yellow	01
red	10

$$L_{A1} = S_1$$
$$L_{A0} = \overline{S_1} \cdot S_0$$
$$L_{B1} = \overline{S_1}$$
$$L_{B0} = S_1 \cdot S_0$$

Schematic:

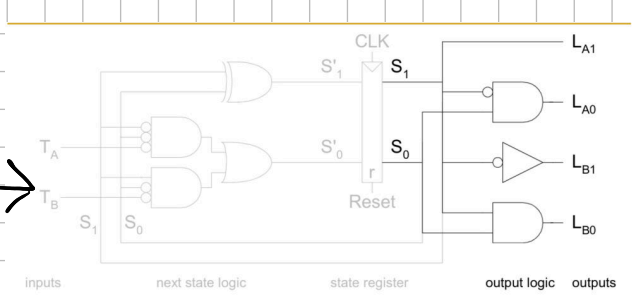
12





$$S'_1 = S_1 \text{ xor } S_0$$

$$S'_0 = (\overline{S_1} \cdot \overline{S_0} \cdot \overline{T_A}) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B})$$



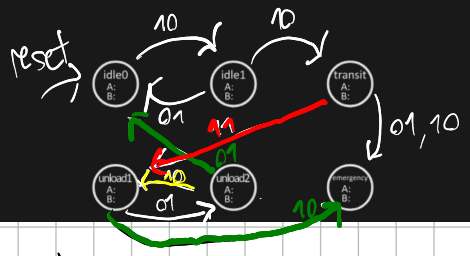
$$L_{A1} = S_1$$
$$L_{A0} = \overline{S_1} \cdot S_0$$
$$L_{B1} = \overline{S_1}$$
$$L_{B0} = S_1 \cdot S_0$$

Practical Exam Example:

Note: I see it as very unlikely that you will have to draw up a schematic, everything else is important!

from FS22:

(a) [25 points] Complete the Moore-type FSM below by (1) drawing the transition edges between the states (including reset), (2) specifying the edges' respective input bits, and (3) specifying the output bits of each state. Any input for which no outgoing edge is specified is assumed as a self loop. The 6 given states are sufficient, do not draw additional states.
Note: Passengers sometimes slip in or out through incorrect doors, or even through closed doors. Your FSM must correctly handle such cases.



outputs: idle 0 → 10, idle 1 → 10, transit → 00, unload1 → 01, unload2 → 01, emergency → 11

2 Finite State Machines [50 points]

The Polybahn from Central to Polyterasse has broken down! To fix it you need to design a finite state machine that controls the Polybahn's two doors A and B.

The Polybahn should operate as follows:

- Initially the Polybahn is empty and **idle**, and passengers can enter through door A.
- The Polybahn is full when it carries 2 passengers.
- When it is full and idle, the Polybahn goes into **transit** to the other station with both doors closed.
- After reaching the station, the Polybahn will **unload** all passengers through door B, while door A is still closed.
- After the last passenger has exited the Polybahn, door B closes and the Polybahn becomes **idle**.
- Should (1) a passenger fall out of the Polybahn during **transit**, or (2) the Polybahn become overfull (≥ 3 passengers) at any point, it stops in **emergency** mode, where it opens all doors and remains (unless reset to the initial **idle** state).

The FSM receives two input bits, with the following meaning:

Input	Meaning
00	no change
01	exactly one passenger left
10	exactly one passenger entered
11	the Polybahn arrived in a station

The FSM produces two output bits: The first bit, A, holds door A open when it is 1. The second bit, B, holds door B open when it is 1.

(a) [25 points] Complete the Moore-type FSM below by (1) drawing the transition edges between the states (including reset), (2) specifying the edges' respective input bits, and (3) specifying the output bits of each state. Any input for which no outgoing edge is specified is assumed as a self loop. The 6 given states are sufficient, do not draw additional states.
Note: Passengers sometimes slip in or out through incorrect doors, or even through closed doors. Your FSM must correctly handle such cases.

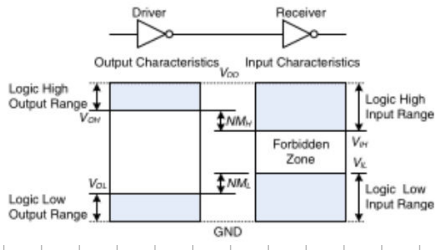
Timing:

Electrical Basics:

lowest voltage in a system → ground/GND (usually 0V)

highest voltage in a system → V_{DD}, comes from PSU

Logic Levels:



→ first gate: driver, second gate: receiver

→ driver produces logical LOW (0) → output between 0 and V_{OL} (output low)

HIGH (0) → output between V_{OH} (output high) and V_{DD}

→ receiver gets input between 0 and V_{IL} (input low) → reads logical LOW

input high → V_{IH} and V_{DD} → reads logical HIGH

→ forbidden zone: between V_{IL} and V_{IH} (caused by defects or noise)

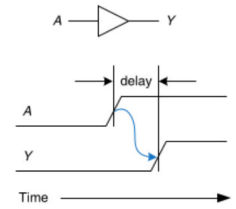
→ Noise Margins: $NM_L = V_{IL} - V_{OL}$ (low margin) $\quad NM_H = V_{OH} - V_{IH}$ (high margin) → reasoning: we need output highs > input high, output low < input low, to correctly interpret outputs (with noise)

→ **Static Discipline:** to avoid forbidden zone → given a log. valid input, any circuit element produces log. valid outputs

→ we can arbitrarily choose V_{DD}, but communicating gates must have compatible logic levels

→ we group gates into logic families like CMOS → all gates in CMOS adhere to static discipline with each other

Combinational Timing: we want to optimize the speed of a circuit



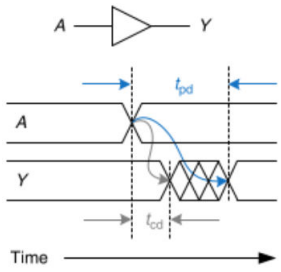
→ input change and output change have a delay for any circuit element

→ timing diagrams like this show the transient response of circuits

→ rising edge → logic LOW → HIGH, falling edge → logic HIGH → LOW

→ blue arrow indicates that rising edge of A causes rising edge of Y

→ delay is measured from 50% point (when signal is 50% between LOW/HIGH) of input to 50% point of output



t_{pd} → propagation delay → max. time from the input change to the output's final value

t_{cd} → contamination delay → max. time from the input change to the output value change

Critical path: slowest path with the most changes

Short path: fastest path with the least changes

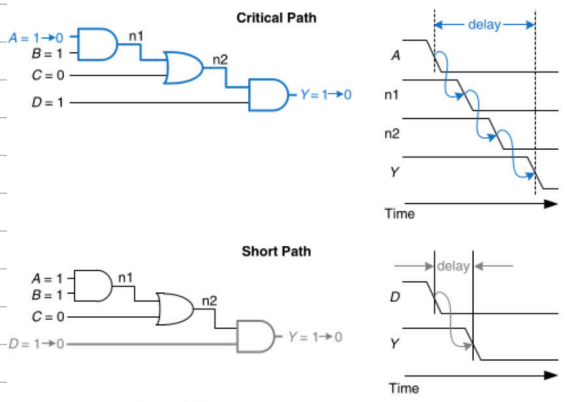


Figure 2.69 Critical and short path waveforms

→ propagation delay of combinational circuit: sum of t_{pd} on critical path

→ contamination delay of comb. circuit: sum of t_{cd} on short path

Glitches: single input transition causes multiple output transitions

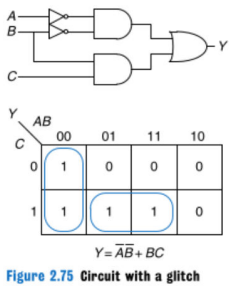
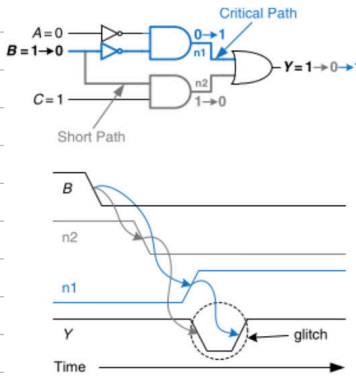


Figure 2.75 Circuit with a glitch



→ short path goes through AND and OR

→ critical goes through AND, OR, NOT

→ falling edge on B → n_2 falls on short path before n_1 rises on crit.

→ until n_1 rises, two inputs to OR gate are 0 → Y drops to 0

→ when n_1 does rise → Y comes back to 1 → that was our glitch

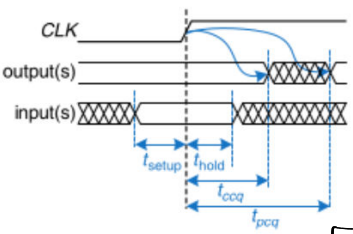
→ glitches are fine when we wait for t_{pd} before using the output, but here, we could prevent it by adding a redundant gate

Sequential Timing: remember how in a flip-flop, D is sampled on the clock edge?

→ an apt comparison to what happens in circuits is how you have to hold a camera still for a while before a shot and after (aperture time) → our aperture time consists of the setup (t_{setup}) and hold (t_{hold}) time

→ sequential circuits have a clock-to-Q contamination (t_{ccq}) and propagation (t_{pcq}) delay → similar to combinational circuits, they are the fastest and slowest delays

→ for the circuit to sample its input correctly, the input must have been stable for t_{setup}

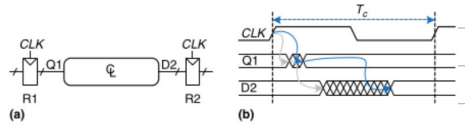


before the clock edge and t_{hold} after ⇒ aperture time = $t_{setup} + t_{hold}$

Dynamic Discipline: the inputs of a sync. circuit **must** be stable during the aperture time

→ sample inputs that are not changing

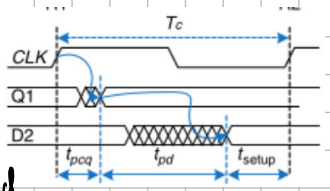
System Timing: clock period T_c = time between rising edges of a clock, $1/T_c = f_c$ → clock frequency, which is measured in Hz (cycles per second)



→ in this image, we try to **calculate** the clock period

→ R_1 produces Q_1 on the rising edge, they go through a block of comb. logic that produces D_2, which goes to R_2

→ gray arrows → **contamination delay**, blue arrows → **propagation delay**



Setup Time Constraint:

→ D_2 must be settled no later than before t_{setup}

→ we know that $T_c \geq t_{pcq} + t_{pd} + t_{setup}$ → can be rearranged to $t_{pd} \leq T_c - (t_{pcq} + t_{setup})$ (sequencing overhead)

this equation is called setup time constraint,

because the comb. logic block cannot meaningfully compute D_2 during t_{pcq} and t_{setup}

→ if t_{pd} is too long, D_2 may not be at its final value when needed → malfunction → solution: increase T_c or redesign CL

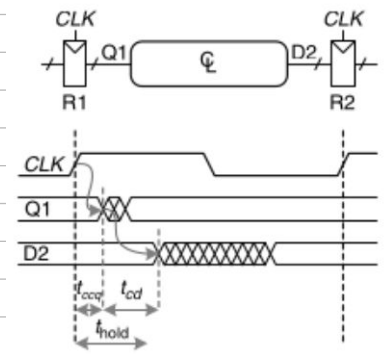
Hold Time Constraint:

→ the designer usually **doesn't** have control over t_{ccq}, t_{pcq}, t_{hold}, t_{setup}, or T_c (depends on manufacturers ← dictated internally)

→ hold time constraint: D_2 must not change for t_{hold} after rising edge, but it can change $t_{ccq} + t_{cd}$ after rising edge ⇒ $t_{ccq} + t_{cd} \geq t_{hold}$

⇒ $t_{cd} \geq t_{hold} - t_{ccq}$ (min-delay constraint)

↑ t_{ccq} and t_{hold} outside our control



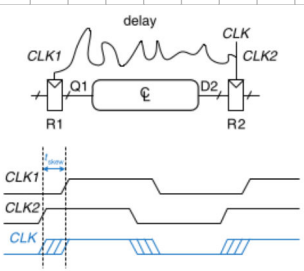
Example: 2 flip-flops: no comb. logic between → $t_{hold} \leq t_{ccq}$, in practice, $t_{hold} = 0$ (or any chain of seq. logic elements)

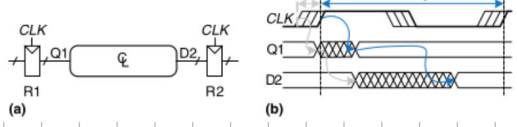
→ hold time violations **cannot be fixed** without redesigning the circuit (by increasing contamination delay) → cannot be fixed by adjusting clock period → **very serious** for real circuits

Clock skew (this came up in the lecture, so we must examine it)

→ clock edges may vary (clock skew), caused by differing clock-register wire length, noise, and using both gated and ungated clocks

→ here, CLK_2 is early because of the routing of the clock wire between R_1 and R_2

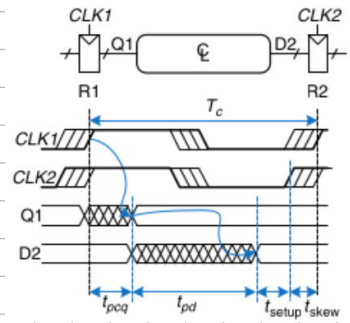




→ clock may arrive **up to t_{skew}** earlier

→ effect on setup time constraint:

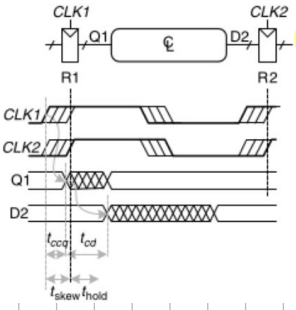
→ worst case: R_1 receives latest skewed clock, R_2 receives earliest skewed clock because comb. logic propagation has least time



→ data still has to setup and propagate before sampling ⇒ $T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew}$

⇒ $t_{pd} \leq T_c - (t_{pcq} + t_{setup} + t_{skew})$ → skew added to constraint

Effect on hold time constraint:



→ worst case: R_1 gets earliest skewed clock, R_2 receives latest skewed clock

⇒ data must not arrive until hold time after late clock

⇒ $t_{ccq} + t_{cd} \geq t_{hold} + t_{skew}$ ⇒ $t_{cd} \geq t_{hold} + t_{skew} - t_{ccq}$

→ overall, clock skew increases both setup and hold time

Metastability: result of violating dyn. discipline (in flip-flops) ⇒ output can take on voltage between 0 and V_{DD} in forbidden zone → metastable state

→ flip-flop will resolve output to 1 or 0, but resolution time is unbounded

Resolution time: when a flip-flop input changes at random time during clock cycle, t_{res} (resolution time is a random variable) (Ahh flashbacks! )

→ input changes outside aperture → $t_{res} = t_{pcq}$, input changes within aperture → t_{res} can be longer)

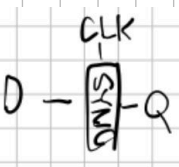
Equation for t substantially longer than t_{pcq} → $P(t_{res} > t) = \frac{T_0}{T_c} e^{-\frac{t}{\tau}}$ → T_0 and τ are characteristic to flip-flop

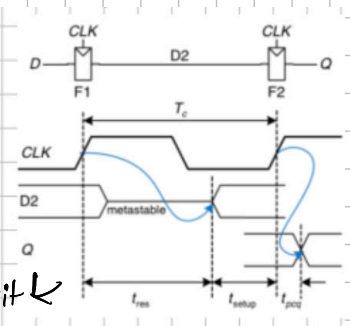
→ $\frac{T_0}{T_c}$ → prob. that input changes at a bad time

Synchronizers:

→ asynchronous inputs like human inputs are inevitable in larger systems → needs handling to prevent metastability

→ a synchronizer receives async. input D and CLK, produces Q in bounded time, the output has a valid logic level with near certainty (if D stable, Q = D, else choose HIGH or LOW)





synchronizer circuit ↑

→ in the image, F_1 samples D on the rising edge of CLK (during aperture, momentarily metastable)

→ Over the clock period, D2 resolves, F_2 samples D2, we get a good output Q

→ synchronizer fails when Q becomes metastable → $t_{res} > T_c - t_{setup}$

→ $P(\text{failure}) = \frac{T_0}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}}$, $P(\text{failure/sec}) = N \frac{T_0}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}}$ (N = times D changes per second)

→ mean time between failures (MTBF) usually measures system reliability ⇒ $MTBF = \frac{1}{P(\text{failure/sec})} = \frac{T_c e^{\frac{T_c - t_{setup}}{\tau}}}{N T_0}$

Some notes on parallelism (pipelining will be introduced)

Token → group of inputs processed to produce group of outputs

Latency → time for 1 token to pass from start to end

Throughput → number of tokens per unit time

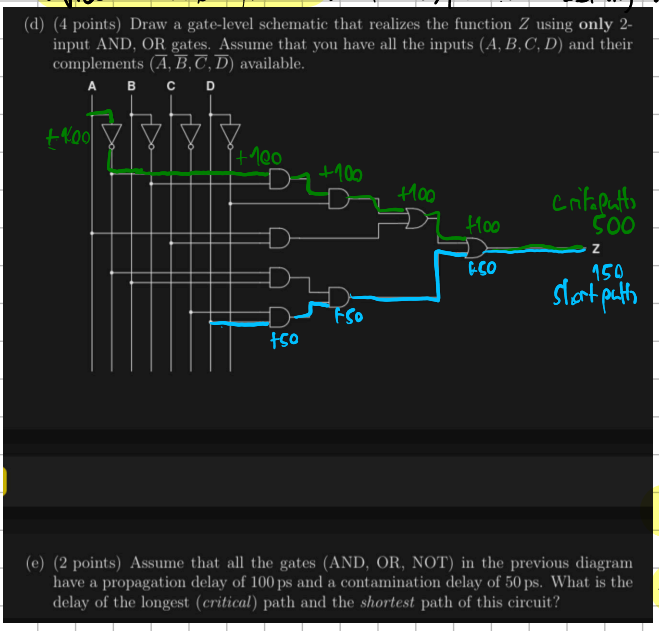
Spatial parallelism: multiple copies of hardware, Temporal parallelism (pipelining): task broken into stages, multiple tasks spread across stages → different task in each stage at once

→ assume a latency L for a nonparallel task with a throughput of 1/L. With N copies of HW, it becomes N/L, which is the ideal case for temporal par. too, but in practice, you get $1/L_1$ ← longest latency for a step

→ Exam practicality: Prof. Mattu hasn't made a timing task yet, but they do come up in older exams

Nice trivial question: (F'15), we're looking at e)

→ in general, I would focus on the basics, less on skew/synchronizers



next section: Verilog!

→ this may be asked: Verilog is not a programming language, it is a HDL (hardware description language) to allow for synthesis of digital circuits, but it often features similar syntax

→ a module is a HW block that needs defined inputs/outputs

→ Behavioral Verilog: using logical operators to describe what a module does

→ Structural Verilog: gate-level description at the bottom level, at a higher level it is about building modules using instances of modules

→ you start with name and listing I/O

→ the assign LHS = RHS statement is used for CL outside always statements

```
module sillyfunction (input a, b, c,
                      output y);

  assign y = ~a & ~b & ~c |
             a & ~b & ~c |
             a & ~b &  c;

endmodule
```

→ ~ → NOT, & = AND, | → OR, ~& → NAND, ~| → NOR, ^ → XOR, ~^ → XNOR

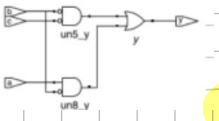
→ Verilog signals (individual bits of variables) can be set to 0, 1, z (floating), x (unknown)

Verilog Operator	Name	Functional Group
[]	bit-select or part-select	
()	parenthesis	
!	logical negation	logical
~	negation	bit-wise
&	reduction AND	reduction
|	reduction OR	reduction
~&	reduction NAND	reduction
~|	reduction NOR	reduction
^	reduction XOR	reduction
~^ or ^~	reduction XNOR	reduction
+	unary (sign) plus	arithmetic
-	unary (sign) minus	arithmetic
{}	concatenation	concatenation
{{}}	replication	replication
*	multiply	arithmetic
/	divide	arithmetic
%	modulus	arithmetic
+	binary plus	arithmetic
-	binary minus	arithmetic
<<	shift left	shift
>>	shift right	shift
>	greater than	relational
>=	greater than or equal to	relational
<	less than	relational
<=	less than or equal to	relational
==	case equality	equality
!=	case inequality	equality
&	bit-wise AND	bit-wise
^	bit-wise XOR	bit-wise
|	bit-wise OR	bit-wise
&&	logical AND	logical
||	logical OR	logical
?:	conditional	conditional

→ Simulation: checking modules for logical correctness

→ Synthesis: HDL code is transformed to a netlist describing hardware produced

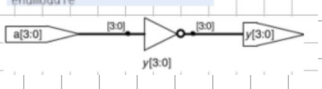
→ code may be simplified during synthesis, there is a schematic output like so:



→ a top module is the main piece of hardware, can consist of CL blocks, registers, FSMs

→ CL: bitwise operators act on single-bit signals or multi-bit busses

```
module inv (input  [3:0] a,
            output [3:0] y);

  assign y = ~a;
endmodule
```



a[3:0] ← 4-bit bus, given in little-endian order (a[3] is MSB, a[0] LSB)

a[0:3] ← big-endian, also permitted

→ just be consistent

```
module gates(input  [3:0] a, b,
             output [3:0] y1, y2,
                          y3, y4, y5);

  /* Five different two-input logic
     gates acting on 4 bit busses */
  assign y1 = a & b;     // AND
  assign y2 = a | b;     // OR
  assign y3 = a ^ b;     // XOR
  assign y4 = ~(a & b); // NAND
  assign y5 = ~(a | b); // NOR
endmodule
```

→ you know the code, the assign statements we saw are continuous, end with ;

→ when inputs on RHS change, LHS recomputed

→ Verilog comments are exactly like Java

→ white space is irrelevant, module names can't begin with numbers, vars. are case-sensitive

reduction operators → using the operators we just introduced in a unary way

```
module and8(input [7:0] a,
            output      y);

  assign y = &a;

  // &a is much easier to write than
  // assign y = a[7] & a[6] & a[5] & a[4] &
  //            a[3] & a[2] & a[1] & a[0];
endmodule
```

conditional assignment: you know this as the ternary operator

```
module mux4(input  [3:0] d0, d1, d2, d3,
            input  [1:0] s,
            output [3:0] y);

  assign y = s[1] ? (s[0] ? d3 : d2)
                  : (s[0] ? d1 : d0);
endmodule
```

(i did this for Lab 5)

```
module fulladder(input  a, b, cin,
                 output s, cout);

  wire p, g;

  assign p = a ^ b;
  assign g = a & b;

  assign s = p ^ cin;
  assign cout = g | (p & cin);
endmodule
```

boolean equations for this full adder: $P = A \text{ XOR } B$, $G = A \cdot B$, $S = P \text{ XOR } C_{in}$, $C_{out} = G + P \cdot C_{in}$

→ the continuous assign statements are concurrent → for CL, order doesn't matter

→ wires are declared to hold internal vars. like P and G, they only need to be declared for multi-bit busses but are good practice

Numbers: (the book won't tell you this: variables can have the data types integer and real → Example: real seven = 7.00;)

Generally: number format: N'b value, bases: 'b → binary, 'o → octal, 'd → decimal, 'h → hex (N = size in bits, b = base)

→ size can be omitted, then a number is assumed to have as many bits as its expression

Table 4.5 Verilog AND gate truth table with z and x

&	A: 0	1	z	x
B: 0	0	0	0	0
1	0	1	x	x
z	0	x	x	x
x	0	x	x	x

← you can see x as "should not happen"

Bit Swizzling: `assign y = {c[2:1], {3{d[0]}}, c[0], 3'b101};`

→ { } concatenates busses and bits (and constants) → example: {3{d[0]}} copies d[0] thrice

you can push z, x →

```
module tristate(input  [3:0] a,
                input        en,
                output [3:0] y);

  assign y = en ? a : 4'bz;
endmodule
```

Delays: ignored in synthesis, used in simulation and testing → ensure timing is correct

→ of course, electrical factors in gate delays cannot be considered

```
`timescale 1ns/1ps

module example(input  a, b, c,
               output y);
  wire ab, bb, cb, n1, n2, n3;

  assign #1 {ab, bb, cb} = ~{a, b, c};
  assign #2 n1 = ab & bb & cb;
  assign #2 n2 = a & bb & cb;
  assign #2 n3 = a & bb & c;
  assign #4 y = n1 | n2 | n3;
endmodule
```

→ timescale is necessary in a file i believe, format: 'timescale unit/precision'

→ here: every unit is 1ns, sim. has 1ps precision

→ # indicates number of units of delay → here: and → 2ns delay, or → 4ns delay

→ can be placed in assign, non-blocking, blocking assignments

Structural modeling:

→ the book actually employs bad practice for instantiation:

→ good practice: submodule name (.input1(i1), .output1(o1));
example → mux2 lowmux(.d0(d0), .d1(d1), .s(s[0]), .y(low));

```
module mux2(input  [3:0] d0, d1,
            input        s,
            output [3:0] y);
  tristate t0(d0, s, y);
  tristate t1(d1, s, y);
endmodule
```

you can negate in instantiations

```
module mux4(input  [3:0] d0, d1, d2, d3,
            input  [1:0] s,
            output [3:0] y);

  wire [3:0] low, high;

  mux2 lowmux(d0, d1, s[0], low);
  mux2 highmux(d2, d3, s[0], high);
  mux2 finalmux(low, high, s[1], y);
endmodule
```

→ 3 instances of mux2

→ accessing parts of busses: use array notation following your endian format → d0[7:4] (example)

```
module mux2_8(input  [7:0] d0, d1,
              input        s,
              output [7:0] y);

  mux2 lsbmux(d0[3:0], d1[3:0], s, y[3:0]);
  mux2 msbmux(d0[7:4], d1[7:4], s, y[7:4]);
endmodule
```


18

Sequential Blocks:

```
module flop (input            clk,
             input      [3:0] d,
             output reg [3:0] q);

  always @ (posedge clk)
    q <= d;
endmodule
```

→ register logic is performed using always statements (always @ (sensitivity list) :)

→ signals keep their values unless an event in the sensitivity list occurs

→ usually it is a clock, as in this case → posedge: rising edge, negedge: falling edge

→ we don't use assign statements here, those are for CL blocks outside of always statements

→ instead, we use the nonblocking assignment (<=), which is used for seq. logic and runs concurrently (no order) unlike =

→ all signals on the LHS of always statements must be declared a register → example: output reg [3:0] q

example: resettable register, allowing for both sync. and async. reset (2 modules)

```
module flopr (input            clk,
              input            reset,
              input      [3:0] d,
              output reg [3:0] q);

  // asynchronous reset
  always @ (posedge clk, posedge reset)
    if (reset) q <= 4'b0;
    else       q <= d;
endmodule

module flopr (input            clk,
              input            reset,
              input      [3:0] d,
              output reg [3:0] q);

  // synchronous reset
  always @ (posedge clk)
    if (reset) q <= 4'b0;
    else       q <= d;
endmodule
```

→ multiple elements in an always sens. list are separated by a comma or "or"

→ you will see, we have if statements, for loops, while loops ...

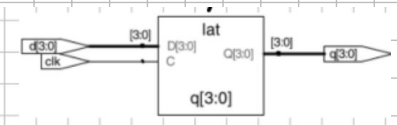
Synchronizer:

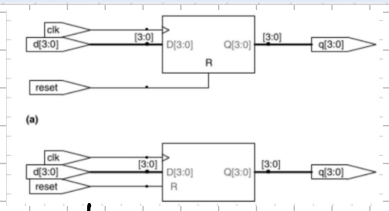
```
module sync (input      clk,
             input      d,
             output reg q);

  reg n1;

  always @ (posedge clk)
    begin
      n1 <= d;
      q <= n1;
    end
endmodule
```

→ begin and end are necessary in any statement or block that makes more than one assignment, good practice otherwise

Latch:

```
module latch (input            clk,
              input      [3:0] d,
              output reg [3:0] q);

  always @ (clk, d)
    if (clk) q <= d;
endmodule
```





→ always @ (clk, d) recomputes statements inside whenever clk or d changes

→ always @ (*) recomputes the statements inside whenever any signal on RHS of = or <= changes

→ blocking assignment (=) used for CL in always blocks, they run sequentially in-order

→ always blocks mostly for sequential blocks, but can behaviorally describe CL if the sens. list responds to all input changes and the body gives outputs in all cases

→ the book calls always blocks "always/process statements"

→ case and if can only be used in always or initial blocks (initial does not synthesize and is used for testbenches)

Verilog

```
module inv (input      [3:0] a,
            output reg [3:0] y);

  always @ (*)
    y = ~a;
endmodule
```

```
module sevenseg (input      [3:0] data,
                 output reg [6:0] segments);
  always @ (*)
    case (data)
      //              abc_defg
      0: segments = 7'b111_1110;
      1: segments = 7'b011_0000;
      2: segments = 7'b110_1101;
      3: segments = 7'b111_1001;
      4: segments = 7'b011_0011;
      5: segments = 7'b101_1011;
      6: segments = 7'b101_1111;
      7: segments = 7'b111_0000;
      8: segments = 7'b111_1111;
      9: segments = 7'b111_1011;
      default: segments = 7'b000_0000;
    endcase
endmodule
```

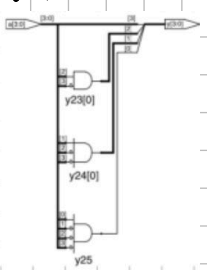
→ case checks value of an input (here data) in base 10

→ if-else statements can be made like in regular programming, just follow the begin/end rules

→ casez includes don't cares from TT's as ?

→ rule in always blocks: don't assign to the same signal more than once

```
module priority_casez (input      [3:0] a,
                       output reg [3:0] y);

  always @ (*)
    casez (a)
      4'b1???: y = 4'b1000;
      4'b01??: y = 4'b0100;
      4'b001?: y = 4'b0010;
      4'b0001: y = 4'b0001;
      default: y = 4'b0000;
    endcase
endmodule
```



19

FSM:

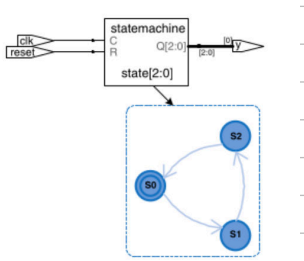
Verilog (divides clk by 3)

```
module divideby3FSM (input  clk,
                     input  reset,
                     output y);

  reg [1:0] state, nextstate;

  parameter S0 = 2'b00;
  parameter S1 = 2'b01;
  parameter S2 = 2'b10;

  // state register
  always @ (posedge clk, posedge reset)
    if (reset) state <= S0;
    else       state <= nextstate;

  // next state logic
  always @ (*)
    case (state)
      S0: nextstate = S1;
      S1: nextstate = S2;
      S2: nextstate = S0;
      default: nextstate = S0;
    endcase

  // output logic
  assign y = (state == S0);
endmodule
```

→ parameter statements are used to define constants and are standard for any FSM state

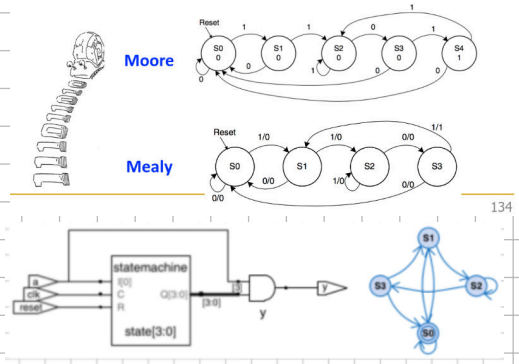
→ default in case needed to fulfill our def. of combinational for next state logic

→ notice the equality comparison == or inequality != with strict versions === and !==



FSM Example 2: Recall the Smiling Snail

- Alyssa P. Hacker has a snail that crawls down a paper tape with 1's and 0's on it
- The snail smiles whenever the last four digits it has crawled over are 1101
- Design Moore and Mealy FSMs of the snail's brain



Verilog (Moore-style)

```
module patternMoore (input  clk,
                     input  reset,
                     input  a,
                     output y);

  reg [2:0] state, nextstate;

  parameter S0 = 3'b000;
  parameter S1 = 3'b001;
  parameter S2 = 3'b010;
  parameter S3 = 3'b011;
  parameter S4 = 3'b100;

  // state register
  always @ (posedge clk, posedge reset)
    if (reset) state <= S0;
    else       state <= nextstate;

  // next state logic
  always @ (*)
    case (state)
      S0: if (a) nextstate = S1;
          else   nextstate = S0;
      S1: if (a) nextstate = S2;
          else   nextstate = S0;
      S2: if (a) nextstate = S2;
          else   nextstate = S3;
      S3: if (a) nextstate = S4;
          else   nextstate = S0;
      S4: if (a) nextstate = S2;
          else   nextstate = S0;
      default:   nextstate = S0;
    endcase

  // output logic
  assign y = (state == S4);
endmodule
```

Verilog (Mealy style)

```
module patternMealy (input  clk,
                     input  reset,
                     input  a,
                     output y);

  reg [1:0] state, nextstate;

  parameter S0 = 2'b00;
  parameter S1 = 2'b01;
  parameter S2 = 2'b10;
  parameter S3 = 2'b11;

  // state register
  always @ (posedge clk, posedge reset)
    if (reset) state <= S0;
    else       state <= nextstate;

  // next state logic
  always @ (*)
    case (state)
      S0: if (a) nextstate = S1;
          else   nextstate = S0;
      S1: if (a) nextstate = S2;
          else   nextstate = S0;
      S2: if (a) nextstate = S2;
          else   nextstate = S3;
      S3: if (a) nextstate = S1;
          else   nextstate = S0;
      default:   nextstate = S0;
    endcase

  // output logic
  assign y = (a & state == S3);
endmodule
```

Parametrizing:

Verilog

```
module mux2
  #(parameter width = 8)
   (input  [width-1:0] d0, d1,
    input              s,
    output [width-1:0] y);
   assign y = s ? d1 : d0;
endmodule
```

default value

depends on width

we are passing in variable to decide module's width

```
module mux4_8 (input  [7:0] d0, d1, d2, d3,
               input  [1:0] s,
               output [7:0] y);

  wire [7:0] low, hi;

  mux2 lowmux (d0, d1, s[0], low);
  mux2 himux (d2, d3, s[1], hi);
  mux2 outmux (low, hi, s[1], y);
endmodule
```

→ you can instantiate like normal - 8 is default

```
module mux4_12 (input  [11:0] d0, d1, d2, d3,
                input  [1:0]  s,
                output [11:0] y);

  wire [11:0] low, hi;

  mux2 #(12) lowmux(d0, d1, s[0], low);
  mux2 #(12) himux(d2, d3, s[1], hi);
  mux2 #(12) outmux(low, hi, s[1], y);
endmodule
```

→ need to instantiate with parameter 12

Testbenches: a HDL module used to test another module, called Device Under Test (DUT)

→ inputs and expected outputs → test vectors

simple testbenches: instantiate DUT, apply inputs in initial block, use blocking assignments and delays, then the user manually verifies correct outputs → simulated like any module, not synthesizable

→ **example:** sillyfunction is a simple 3-input logic function → 8 test vectors

```
module testbench1 ();
  reg a, b, c;
  wire y;

  // instantiate device under test
  sillyfunction dut (a, b, c, y);

  // apply inputs one at a time
  initial begin
    a = 0; b = 0; c = 0; #10;
    c = 1;               #10;
    b = 1; c = 0;        #10;
    c = 1;               #10;
    a = 1; b = 0; c = 0; #10;
    c = 1;               #10;
    b = 1; c = 0;        #10;
    c = 1;               #10;
  end
endmodule
```

→ initial statement: execute at start of simulation (0 time units) // here, we delay by 10 units per input

→ initial doesn't synthesize to hardware, signals assigned to in initial must be reg

self-checking testbench: more like a unit test, less tedious to verify → check against expected outputs, log failures

```
module testbench2();
  reg a, b, c;
  wire y;

  // instantiate device under test
  sillyfunction dut(a, b, c, y);

  // apply inputs one at a time
  // checking results
  initial begin
    a = 0; b = 0; c = 0; #10;
    if (y !== 1) $display("000 failed.");
    c = 1;               #10;
    if (y !== 0) $display("001 failed.");
    b = 1; c = 0;        #10;
    if (y !== 0) $display("010 failed.");
    c = 1;               #10;
    if (y !== 0) $display("011 failed.");
    a = 1; b = 0; c = 0; #10;
    if (y !== 1) $display("100 failed.");
    c = 1;               #10;
    if (y !== 1) $display("101 failed.");
    b = 1; c = 0;        #10;
    if (y !== 0) $display("110 failed.");
    c = 1;               #10;
    if (y !== 0) $display("111 failed.");
  end
endmodule
```

→ $ is a prefix for system tasks, $display prints a message to the simulator console

→ only strict equality works well with x and z

Using a test vector file is more efficient for more vectors:

```
000_1
001_0
010_0
011_0
100_1
101_1
110_0
111_0
```

→ binary text file

→ we use a separate test clock to simulate until file is done

→ $readmemb reads a binary file into the testvectors array,

```
module testbench3();
  reg        clk, reset;
  reg        a, b, c, yexpected;
  wire       y;
  reg [31:0] vectornum, errors;
  reg [3:0]  testvectors[10000:0];
  // instantiate device under test
  sillyfunction dut(a, b, c, y);

  // generate clock
  always
    begin
      clk = 1; #5; clk = 0; #5;
    end

  // at start of test, load vectors
  // and pulse reset
  initial
    begin
      $readmemb("example.tv", testvectors);
      vectornum = 0; errors = 0;
      reset = 1; #27; reset = 0;
    end

  // apply test vectors on rising edge of clk
  always @(posedge clk)
    begin
      #1; {a, b, c, yexpected} =
          testvectors[vectornum];
    end

  // check results on falling edge of clk
  always @(negedge clk)
    if (~reset) begin // skip during reset
      if (y !== yexpected) begin
        $display("Error: inputs = %b", {a, b, c});
        $display("  outputs = %b (%b expected)",
                 y, yexpected);
        errors = errors + 1;
      end
      vectornum = vectornum + 1;
      if (testvectors[vectornum] === 4'bx) begin
        $display("%d tests completed with %d errors",
                 vectornum, errors);
        $finish;
      end
    end
endmodule
```

of 4 bit length — up to 10001 vectors

$readmemh does the same for hex files

→ wait one unit of time after rising edge to set new vector and expected output using bit swizzling

→ on falling edge, check if reset not active, error messages if output doesn't match expectation

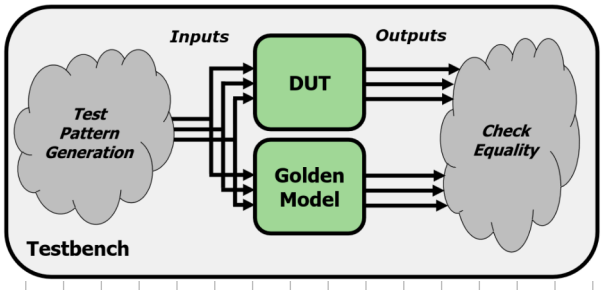
→ $display has this syntax too: $display("custom message %b", {a,b,c})

→ %b prints the incoming number in binary, %d in decimal

→ $finish terminates simulation

Automatic Testbenches:

Golden Models: ideal circuit, may be written in other language



→ we have to automatically generate test vectors → sequential coverage or randomization?

Pseudocode →

```
module testbench1();
  ... // variable declarations, clock, etc.

  // instantiate device under test
  sillyfunction dut (a, b, c, y_dut);
  golden_model gold (a, b, c, y_gold);

  // instantiate test pattern generator
  test_pattern_generator tgen (a, b, c, clk);

  // check if y_dut is ever not equal to y_gold
  always @(negedge clk)
  begin
      if(y_dut !== y_gold)
          $display(...)
  end
endmodule
```

→ this is fully automated, scalable, even timing can be compared with a golden timing model

→ challenges: creating a golden model, test generation, brute forcing tests is inefficient

Von Neumann Model, Processor Design:

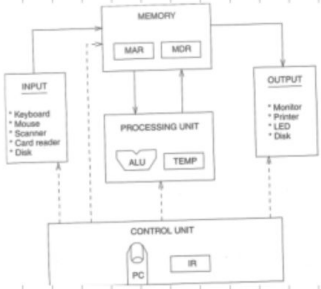
Von Neumann Model:

(smallest pieces of work)

→ a program consists of instructions and is contained in memory, the control unit orders instructions

Memory: if a memory has 2^{28} distinct locations and each can store 8 bits,

it has an address space of 2^{28} locations and addressibility of 8 bits → byte addressable, some ISA's word-addressable



→ reading it: place address in MAR (Memory Access Register) → "interrogate memory"

→ the information stored in that MAR location is then placed in the MDR (Memory Data Register)

000	
001	
010	
011	
100	00000110
101	
110	00000100
111	

→ writing to mem.: put the address in MAR, value in MDR

→ interrogate mem. with WE = 1 (write enable) → MDR info written to the location we wrote into the MAR

→ MAR locations are permanent, what's inside them isn't

Processing Unit: (PU)

→ the simplest PU is an ALU with ADD, SUBTRACT, basic logic

→ the size of quantities processed in bits is the word length, the element is called a word

→ a register file (RF) stores results temporarily (near the ALU) → LC-3 has 8 16-bit registers for this

I/O devices: peripherals like keyboards and displays

Control Unit: contains Instruction Register (IR) that contains the current ins., then a register that contains the next ins.'s address (program counter, abbreviated to PC)

LC-3 as a Von Neumann machine:

→ filled-in arrowheads → data flows using the signal/bus

→ not filled-in → control signals, no data flows

Memory: our MAR (because of an address space of 2^{16} locations) and MDR (because of 16-bit words) are 16-bit

I/O: the KBDR (keyboard data reg.) contains the ASCII of struck keys, the KBSR contains their status

→ the monitor likewise has a Display Data Register and Display Status Reg. for the same purposes

Processing Unit: consists of ALU and RF

Control Unit: FSM directs activity, processing is done cycle-by-cycle (CLK), IR is an input for the FSM, the PC saves the next ins.

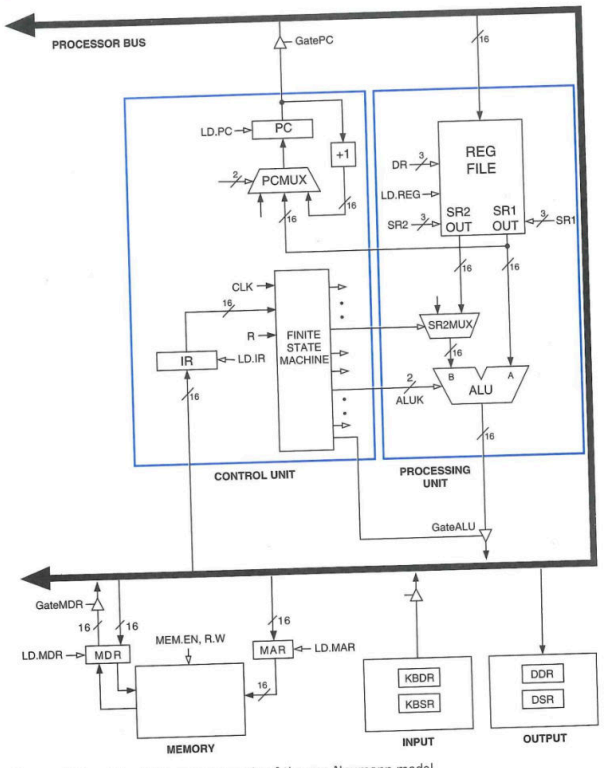


Figure 4.3 The LC-3 as an example of the von Neumann model

→ The FSM outputs the 2-bit opcode ALUK, and control signals like GateALU (determines if ALU output pushed onto processor bus)

Instruction Processing:

→ an instruction consists of an opcode (describing what is done) and operands (where it is done)

22